

Operating Systems (OS)

(Non) Preemptive Scheduling and Scheduling Algorithms

Prof. Dr.-Ing. Jörg Abke



TH Aschaffenburg
university of applied sciences

Copyright Hinweise

© Jörg Abke

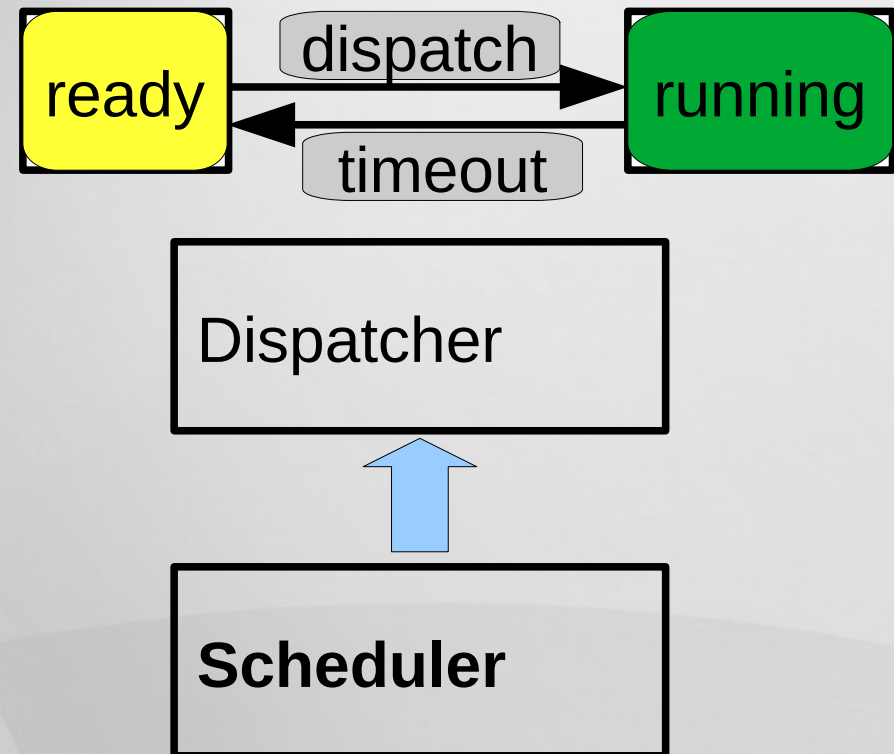
Dieses Werk ist urheberrechtlich geschützt. Alle Rechte, insbesondere Vervielfältigung, Verbreitung oder Übersetzung liegen beim Autor. Kein Teil des Werkes darf ohne Genehmigung des Autors, auch nicht für Unterrichtszwecke, fotokopiert oder durch andere Verfahren reproduziert oder verbreitet werden. Ebenfalls ist die Einspeicherung in EDV-Anlagen ohne Genehmigung des Autors nicht gestattet. Jeder Studierende, der an der entsprechenden Lehrveranstaltung des Autors an der Technischen Hochschule Aschaffenburg teilnimmt, ist berechtigt, einen Ausdruck dieses Werkes anzufertigen und für die Zwecke seines Studiums zu verwenden. Eine weitergehende Verwendung bedarf in jedem Falle der ausdrücklichen Zustimmung des Autors. Gebrauchsname, Handelsnamen, Warenbezeichnungen etc. werden in diesem Werk ohne Gewährleistung der freien Verwendbarkeit benutzt. Texte, Abbildungen, Programme und technische Angaben wurden sorgfältig erarbeitet. Trotzdem sind Fehler nicht völlig auszuschließen. Der Autor weist darauf hin, dass er für Fehlerfreiheit keine Gewähr und für eventuelle Folgen keine Haftung übernimmt.

Dispatcher

- Execution of Context Switch
 - Saving context of timedout process
 - Import of registers' values from saved context of process, which has to be dispatched

Scheduler

- Determination of the point in time for
 - changing process' states (by the dispatcher)
 - execution order



Criteria for Scheduling Processes

CPU utilization / CPU load

- **Response Time**
- **Turnaround time**
- **Waiting time**
 - **Real-time behaviour (meeting response deadlines)**
 - **Data throughput**
- **Overhead**
- **Fairness**
-

Challenge

- No scheduling strategy can meet all criteria optimally
- A Compromise needs to be found

Main Criteria from Processes' Perspective

- **Arrival Time (AT):** Time when process arrives in ready state
- **Completion Time (CT),** Time when process completes its execution
- **CPU time, aka Burst Time (BT):** timespan which a process needs for execution (in running state)
- **Turn Around Time (TAT), also Runtime, or „lifetime“ :** time period from arrival up to termination of a process
$$TAT = CT - AT$$
- **Waiting time (WT):** Time of a process from creation but outside of state running
$$TAT = WT + BT$$

Non-Preemptive Scheduling

- **Categories of scheduling strategy: Non-Preemptive or Preemptive Scheduling**
- **Non-preemptive**
 - A process in running state keeps on running as long as it
 - finishes
 - voluntarily resigns (returns the CPU use)
 - Problem: CPU is occupied as long as the process wants
 - **Examples:** Windows 3.x , MacOS 8/9, Win 95/98/ME (16bit proc.)

Example: Effects on Execution Order (Source: Chr. Baun, Operating Systems)



- **Process PA, PB, Arrival Time = 0**
- **PA: 27 ms CPU time (BT), PB: 3 ms CPU time (BT)**

Order	TAT (ms)	WT (ms)	Avg. TAT (ms)	Avg. WT (ms)
PA → PB	PA: 27	PA: 0	$57/2 = 28,5$	$27/2 = 13,5$
	PB: 30	PB: 27		
PB → PA	PA: 30 PB: 3	PA: 3 PB: 0	$33/2 = 13,5$	$3/2 = 1,5$

Calculation Tasks to Do

- **Non-Preemptive Scheduling, 3 Processes, all Arrival Time: 0**
- **PA: 15 ms CPU time (BT),**
- **PB: 5 ms CPU time (BT),**
- **PC: 12 ms CPU time (BT)**

- **Please calculate for all possible execution orders**
 - Turn Around Time and average Turn Around Time
 - Waiting Time and average Waiting Time

- **Use the tabular on the next slide**

Solution of Calculations



Order	TAT (ms)	WT (ms)	Avg. TAT (ms)	Avg. WT (ms)

3 Processes, all Arrival Time: 0
PA: 15 ms CPU time (BT),
PB: 5 ms CPU time (BT),
PC: 12 ms CPU time (BT)

Preemptive Scheduling

- **Preemptive**
 - A process in running state may be assigned to another state regardless
 - if it is already finished
 - Voluntarily resigns
 - Thus, the process is *interrupted* and *paused*
 - **Drawbacks:** Overhead for context switching, implementation effort, starvation for low priority processes (→ see later), concurrency issues (→ see later)
 - **Examples:** Linux, Windows 95/98/ME (32 bit proc.), Win NT and higher, MacOS X, FreeBSD

Scheduling Methods (Selection)

- **Non-Preemptive Scheduling**
 - **First Come, First Served (FCFS)**
 - **Shortest Job First (SJF)**
 - **Priority Scheduling** (non-preemptive)
 - **Earliest Deadline First (EDF)** (non-preemptive)
- **Preemptive Scheduling**
 - **Shortest Remaining Time First (SRTF)**
 - **Round Robin (RR)**
 - **Priority Scheduling** (preemptive)
 - **Earliest Deadline First (EDF)** (preemptive)
- **Multilevel Scheduling**

First Come First Served (FCFS)

- **First In First Out (FIFO) Principle**
 - Processes run in the order of appearance
- **Simplest scheduling algorithm**
 - Easy to implement with a single FIFO / Queue
- **Suitable for batch processing**
- **Fair method**
 - All Processes are executed
- **Average Waiting Time (AWT) might be high**
- **Used for Linux non-preemptive *real time* processes**

Example for FCFS Scheduling

- Arrival Time of Proc A = 0, B = 2ms, C = 5ms
- Process A: 5 ms | Process B: 3 ms | Process C: 8 ms
- Execution order:

- Waiting Times (WTs):

Process	A	B	C
Waiting Time			

- Average Waiting Time =

- Turn Around Times (TATs)

Process	A	B	C
Turn Around Time			

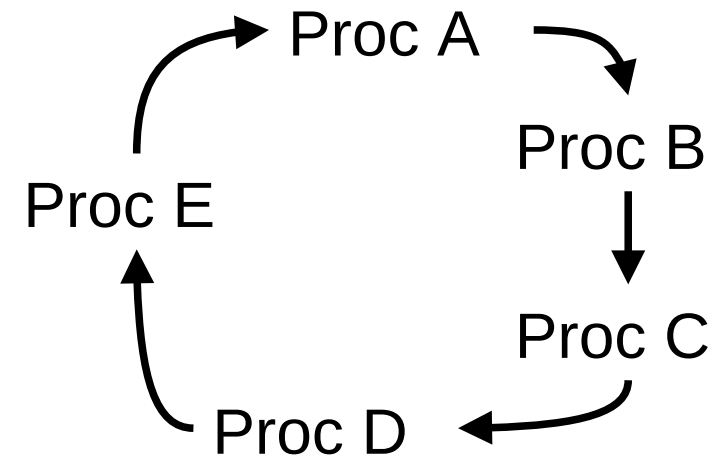
- Average TurnAround Time =

Round Robin (RR) - Time Slicing Algorithm

- dt. Zeitscheibenverfahren
- For time-sharing and interactive systems
- Preemptive method
- Fair method
- Starvation is prevented
- Each process gets a fixed time quantum

Cyclic method:

- All processes in ready state in a circular queue
- If a process does not finish within its quantum, it is preempted and moved to back of the queue
- Scheduler repeats this cycle continuously



Example for Round Robin

- Arrival Time of Proc A = 0ms, B = 3ms, C = 3ms
- Process A: 24ms | Process B: 3ms | Process C: 3 ms, Time Quantum = 4ms
- Execution order:

- Waiting Times (WTs):

Process	A	B	C
Waiting Time			

- Average Waiting Time =

- Turn Around Times (TATs)

Process	A	B	C
Turn Around Time			

- Average TurnAround Time =

Pros and Cons of Round Robin

- **Time Quantum $\rightarrow \infty$: behaviour like FCFS**
- **Short time quantum: context switch overhead**
 - $\rightarrow$ decision of time quantum is essential on system performance**

Priority based Scheduling

- Each Process has a **priority** value (assigned by OS)
 - **Fixed/static**
 - **dynamic** values, adjustment from time to time
(→ Multilevel Feedback Scheduling)
- Process with **highest priority** gets **running** state
- Processes with same priority, FCFS is used
- Preemptive or Non-Preemptive
- Risk of starvation for low priority processes → not fair method

Example for Priority-driven Scheduling

- **Arrival times: 0ms**

- **Non-preemptive**

Process	A	B	C	D
CPU Time needed	8ms	4ms	7ms	13ms
Priority	3	15	8	4

- Execution order:

Waiting Times (WTs):

Process	A	B	C	D
Waiting Times in ms				

Average Waiting Time =

Turn Around Times (TATs)

Process	A	B	C	D
TAT				

Average TurnAround Time =

Example for Priority-driven Scheduling

- **Arrival times: 0ms**
- **Preemptive,**
Time Quantum = 5ms

Process	A	B	C	D
CPU Time needed	8ms	4ms	7ms	13ms
Priority	3	15	8	4

- Execution order:

Waiting Times (WTs):

Process	A	B	C	D
Waiting Times in ms				

Average Waiting Time =

Turn Around Times (TATs)

Process	A	B	C	D
TAT				

Average TurnAround Time =

Earliest Deadline First (EDF)

- Processes with the earliest deadline are given the highest priority
- Scheduler selects the process with the earliest deadline
- Adjustment of the process order dynamically whenever..
 - A running process terminates
 - A new process enters running state
- Non-Preemptive variant for batch processing
- Preemptive variant suitable for real-time operating systems
 - used in Linux for *real-time* processes
 - used Real-Time OS

Issues of Above Scheduling Methods

- **Some scheduling methods (SJF, SRTF, LJF, LRTF) not really usable in practice**
 - Prediction of burst time (CPU time) beforehand is tedious or even impossible
 - Statistics from past runs could be used, still models
- **Trade-offs in criteria in methods like Priority based, FCFS, RR, EDF methods**
- **Given solution: Multilevel Scheduling**

Multilevel Feedback Scheduling

- **Preemptive method**
- **Multiple queues with different Priorities and time quantum**
- **Each queue uses Round Robin**
- **A process stays in its queue if it returns from running state voluntarily (e.g. while waiting for input)**
- **A process is assigned to queue with lower prio, if it uses its time quantum completely**

Advantages:

- Preference of newer processes over older ones
- Processes with many input / output are preferred

Multilevel Feedback Scheduling Usage

- **Many modern OS uses variants of this method**
 - Linux (up to kernel 2.4)
 - Mac OS X
 - FreeBSD, NetBSD
 - MS Windows NT
- **Scheduling methods of latest Linux kernels are left out due to time limitations in this course**

Non-Preemptive Scheduling

- First Come, First Served (FCFS)
- Priority Scheduling (non-preemptive)
- Earliest Deadline First (EDF) (non-preemptive)

Preemptive Scheduling

- Round Robin (RR)
- Priority Scheduling (preemptive)
- Earliest Deadline First (EDF) (preemptive)

Multilevel Scheduling